# ¿QUÉ VA EN CADA PAQUETE?

*Chuleta rápida para los proyectos Spring Boot de DWES*

**IDEA CLAVE**

Controller recibe la petición → Service decide qué hacer → Model guarda los datos.
DTO transporta datos, Exception representa errores y Config guarda configuración.

| Paquete | Qué es | Ejemplos | Qué haces ahí |
|---|---|---|---|
| **model** | Las entidades del programa. | Libro, Categoria, Producto, Cliente, Vehiculo... | Aquí van sus atributos: id, nombre, precio, stock, etc. |
| **dto** | Objetos para recibir o devolver datos concretos. | LibroPeticionDto, VehiculoResumenDto... | Úsalo cuando no quieras recibir/devolver exactamente el Model. |
| **service** | La lógica del programa. | buscar, crear, modificar, borrar, validar, calcular... | Normalmente hay una interfaz Servicio y una clase ServicioImpl. |
| **controller** | Los endpoints de la API. | GET, POST, PUT, DELETE; @RequestBody, @PathVariable, @RequestParam... | Recibe HTTP, llama al Service y devuelve la respuesta. |
| **exception** | Los errores controlados del programa. | RecursoNoEncontradoException, DatosNoValidosException, ConflictoException... | También suele estar aquí el manejador global con ProblemDetail. |
| **config** | Configuración de la aplicación. | @ConfigurationProperties, OpenAPI general... | Aquí se leen propiedades de application.properties o se configura Swagger. |

## Cómo pensar cada paquete sin liarte

### model

**“¿Qué cosas existen en mi programa?”**

> Si el ejercicio es una biblioteca: Libro y Categoria. Si es un taller: Cliente, Vehiculo y Reparacion.

*El Model representa los datos que existen.*

### dto

**“¿Qué datos quiero recibir o devolver en esta operación?”**

> Por ejemplo, para crear un libro quizá recibes titulo, autor, ejemplares y categoriaId. Para un resumen quizá devuelves nombreCategoria y totalLibros.

*El DTO es una caja hecha a medida para una petición o respuesta.*

### service

**“¿Qué tiene que hacer realmente el programa?”**

> Buscar un libro, comprobar que existe una categoría, impedir stock negativo, calcular un total, comprobar una matrícula repetida...

*La lógica importante vive aquí, no en el Controller.*

### controller

**“¿Qué URL va a llamar el usuario y qué método HTTP usa?”**

> GET /api/libros, POST /api/libros, PUT /api/libros/{id}, DELETE /api/libros/{id}.

*El Controller es la puerta de entrada de la API.*

### exception

**“¿Qué error quiero lanzar cuando algo va mal?”**

> ID inexistente → 404. Dato inválido → 400. Regla de negocio incompatible → 409.

*Sirve para no devolver errores caóticos y mantener respuestas claras.*

### config

**“¿Hay valores o configuración que no quiero dejar desperdigados por el código?”**

> biblioteca.max-prestamos, alquiler.max-dias, academia.max-matriculas-activas-alumno, información general de OpenAPI...

*Config centraliza valores y configuración.*

## Y fuera de esos paquetes...

| Archivo / carpeta | Para qué sirve |
|---|---|
| Application.java | Clase principal. Desde aquí arrancas Spring Boot. |
| src/main/resources/application.properties | Propiedades como nombre de la aplicación, límites configurables, etc. |
| pom.xml | Dependencias y configuración Maven del proyecto. |
| postman/ | Colección Postman si el ejercicio pide entregarla. |
| README.md | Explicación del proyecto si el ejercicio la pide. |

**FLUJO QUE DEBES RECORDAR**

Petición HTTP
↓
Controller
↓
Service / ServiceImpl
↓
Model y listas en memoria
↓
Service devuelve resultado
↓
Controller devuelve HTTP/JSON

**Si dudas dónde poner algo: endpoint → Controller; lógica → Service; datos → Model; forma especial de datos → DTO.**